

ARCHITECTURE OVERVIEW

Notification Management Service

Backend design: components, data model, state machine, delivery flow, and trade-offs

Attribute	Value
Document	Architecture Overview — Backend Service
Scope	Notification Management Service (NMS) backend
Related documents	docs/scenarios/01-greenfield.md , docs/scenarios/02-brownfield.md , docs/testing-strategy.md
Version / Date	1.0 — September 2026

Contents

1. Purpose and Scope 3

2. High-Level Component View 3

 2.1 Layer responsibilities 4

3. Data Model 5

 3.1 Notable constraints 5

4. Notification Status State Machine 6

 4.1 Status aggregation 6

5. Submission-to-Delivery Sequence 7

 5.1 Key points in the flow 7

6. Async Processing Without an External Broker 8

7. Channel Routing Policy 8

8. Retry and Failure Classification 8

9. Deduplication and Idempotency Boundaries 9

 9.1 Request-level 9

 9.2 Delivery-level 9

10. Security and Data-Handling Notes 9

11. Deliberately Out of Scope 9

1. Purpose and Scope

The Notification Management Service receives alert requests from upstream business and technical systems and delivers notifications to recipients through configurable channels — email, SMS, push, and Slack. It provides asynchronous processing, bounded retry, deduplication, status retrieval, and an audit trail.

This document covers the backend service only. The three graded scenarios (greenfield, brownfield, and ambiguous requirement) are documented separately in [docs/scenarios/](#).

2. High-Level Component View

The service is organized as a hexagonal (ports-and-adapters) architecture, packaged by layer and then by feature. Figure 1 shows the major components and their dependencies.

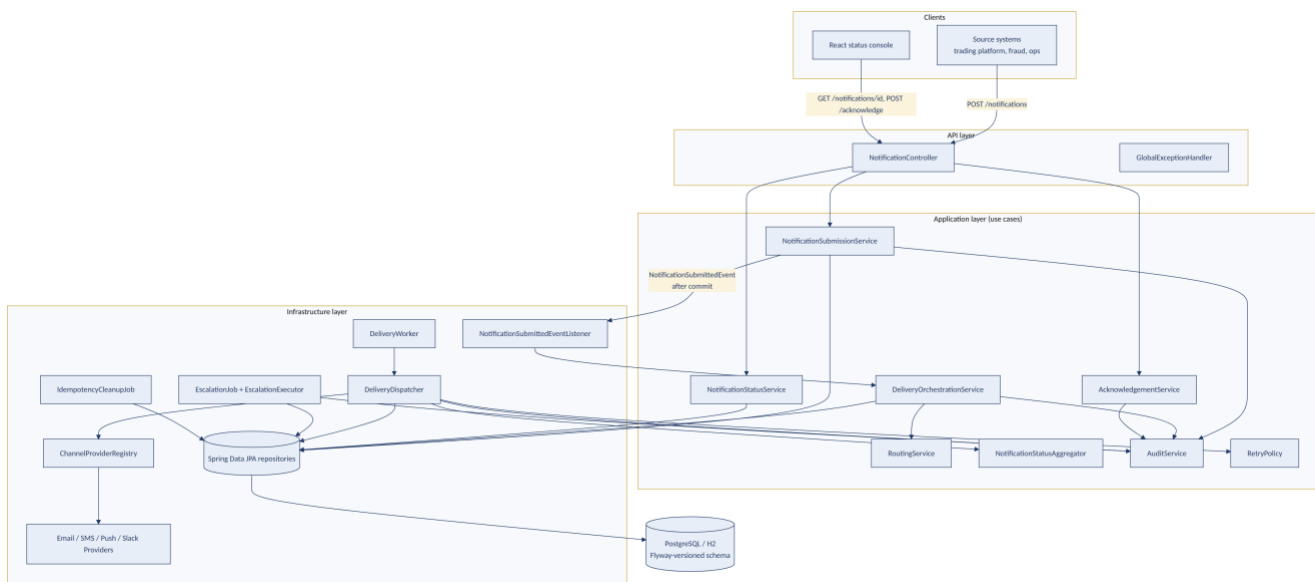


Figure 1 — Component view across the API, application, and infrastructure layers

Why this shape

Delivery-channel integrations, the scheduling mechanism, and the persistence technology are all replaceable without touching business logic. For example, swapping the H2/Postgres polling worker for a Kafka consumer only touches `infrastructure.worker` and `infrastructure.provider`.

2.1 Layer responsibilities

Package	Responsibility	Depends on
api	HTTP contracts: DTOs, controllers, and <code>@RestControllerAdvice</code> error mapping. Bean Validation enforced at the boundary.	application
application	Use cases and business rules — routing policy, retry policy, orchestration, aggregation, acknowledgement. No JPA or HTTP types leak in; commands and results are plain records.	domain, repository interfaces
domain	JPA entities and enums — the persistent model and its invariants (e.g. <code>DeliveryAttempt.isTerminal()</code>).	Nothing (innermost layer)
infrastructure	Adapters: Spring Data repositories, channel providers (Strategy pattern), scheduled workers, configuration.	domain, application interfaces

3. Data Model

Figure 2 shows the persistent schema. All tables are Flyway-versioned and run identically on PostgreSQL (production) and H2 (tests).

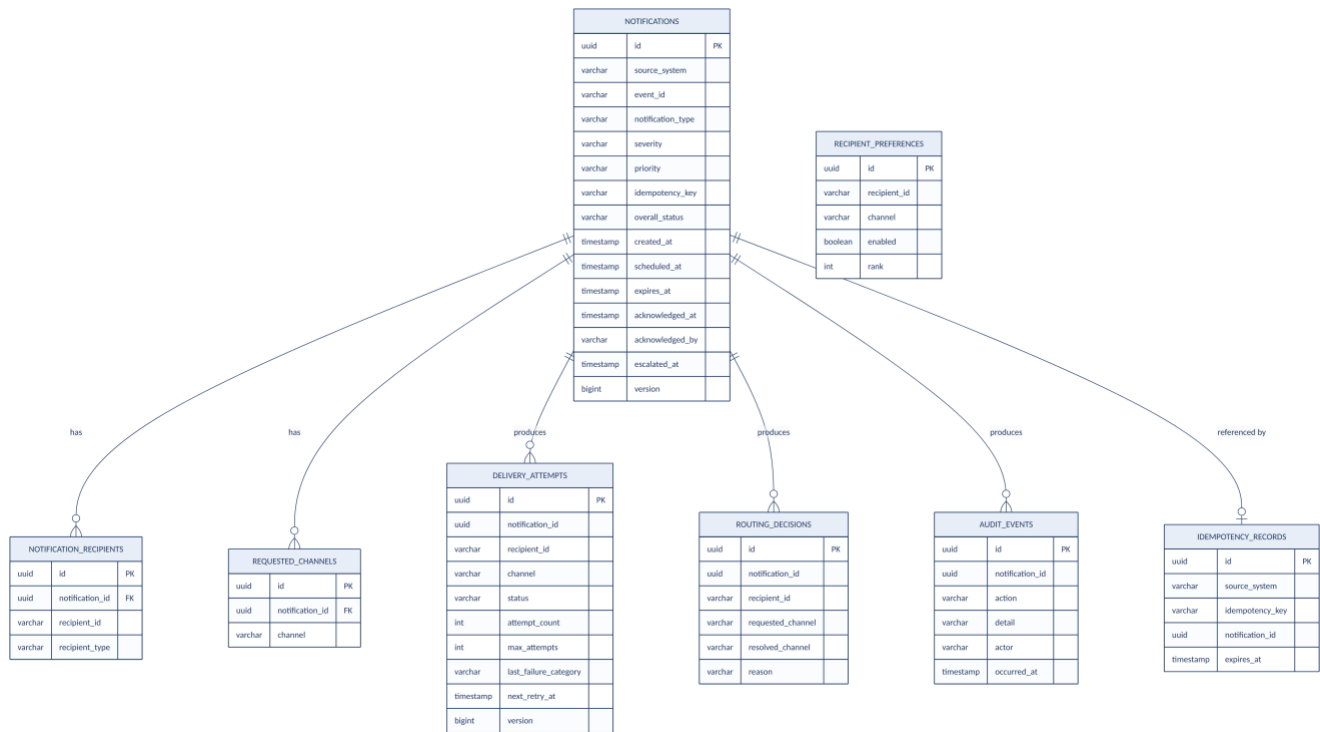


Figure 2 — Entity-relationship diagram

3.1 Notable constraints

- `delivery_attempts` (`notification_id`, `recipient_id`, `channel`) carries a unique constraint. This is the **delivery-level** deduplication boundary and is independent of the request-level `idempotency_records` boundary (see Section 9).
- `delivery_attempts.notification_id`, `routing_decisions.notification_id`, and `audit_events.notification_id` are logical foreign keys, not enforced by the database.
- `notifications.version` and `delivery_attempts.version` provide optimistic locking.
- `idempotency_records.expires_at` implements a 7-day retention window.
- `audit_events.detail` holds structured, non-sensitive metadata only.

Full rationale is documented in `docs/scenarios/01-greenfield.md` and `docs/scenarios/02-brownfield.md`.

4. Notification Status State Machine

A notification moves through the states shown in Figure 3. **ESCALATED** is reachable from any non-terminal in-flight state when a **CRITICAL** notification remains unacknowledged past its deadline.

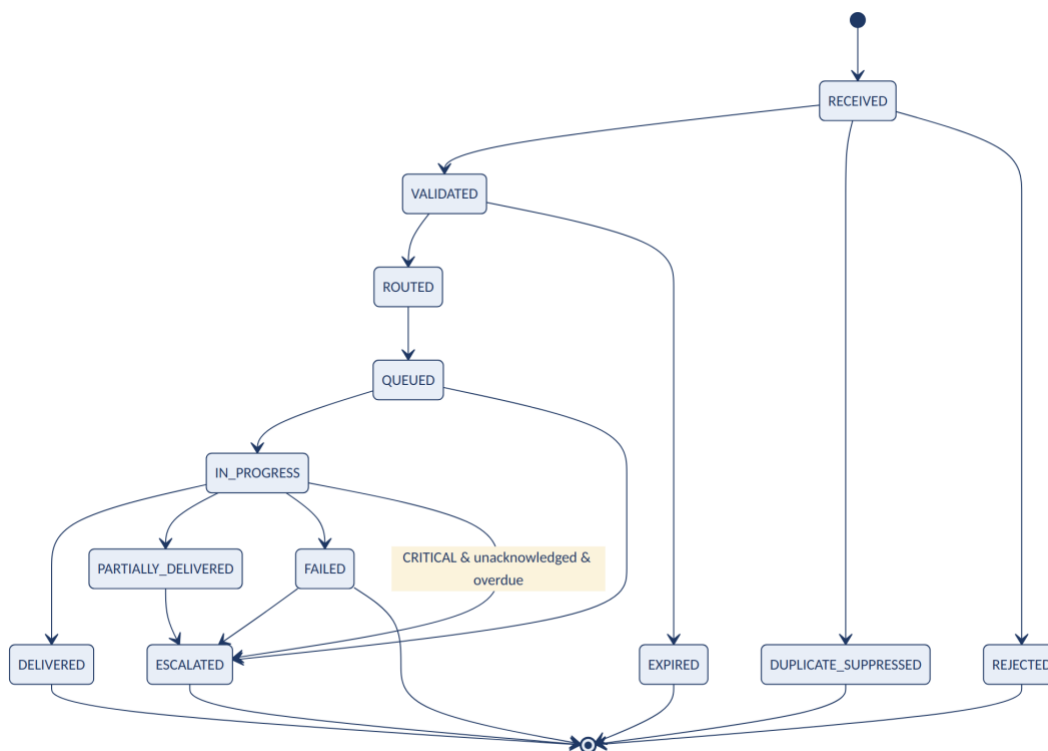


Figure 3 — Overall notification status lifecycle

4.1 Status aggregation

`NotificationStatusAggregator` recomputes **DELIVERED**, **PARTIALLY_DELIVERED**, **FAILED**, and **IN_PROGRESS** from the live set of `DeliveryAttempt` rows after every attempt outcome.

Terminal-for-aggregation states

The aggregator explicitly does **not** overwrite **EXPIRED**, **REJECTED**, or **ESCALATED** once set. See the class Javadoc for why **ESCALATED** in particular needed this protection.

5. Submission-to-Delivery Sequence

Figure 4 traces a request from POST `/api/v1/notifications` through orchestration, polling, dispatch, and status retrieval.

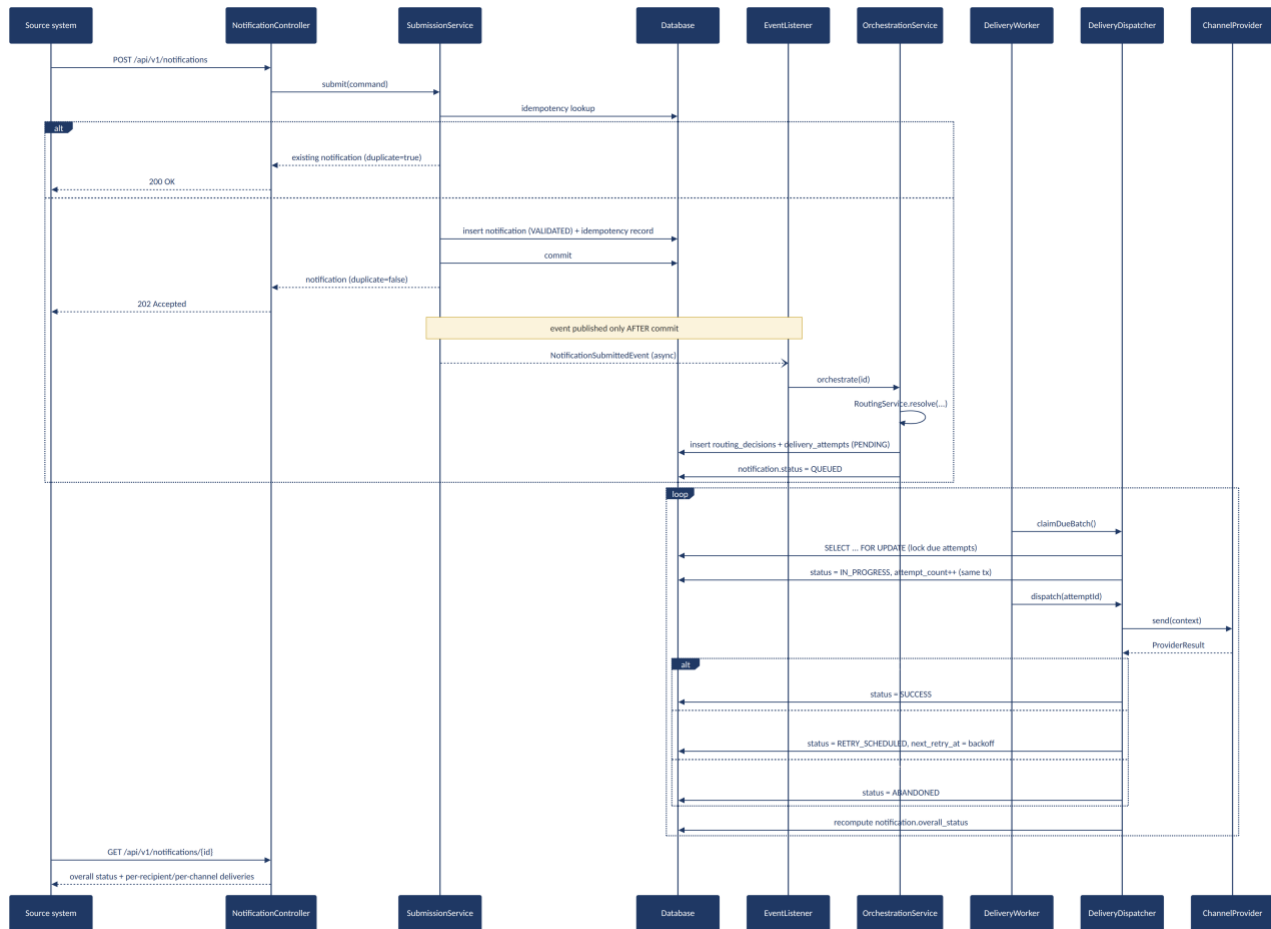


Figure 4 — Submission, orchestration, and delivery sequence

5.1 Key points in the flow

- Idempotency check first.** A repeated `(source_system, idempotency_key)` that has not expired returns the existing notification with `duplicate=true` and HTTP 200; new submissions return 202 Accepted.
- Event after commit.** `NotificationSubmittedEvent` is published only after the submission transaction commits, so the listener never observes an uncommitted row.
- Orchestration.** `DeliveryOrchestrationService` resolves routing, inserts `routing_decisions` and PENDING `delivery_attempts`, and moves the notification to `QUEUED`.
- Claim-then-dispatch.** The worker locks due attempts with `SELECT ... FOR UPDATE`, flips them to `IN_PROGRESS` and increments `attempt_count` in the same transaction, and only then calls the provider.
- Outcome handling.** Success → `SUCCESS`; retryable failure with attempts remaining → `RETRY_SCHEDULED` with a backoff `next_retry_at`; non-retryable or exhausted → `ABANDONED`. The overall notification status is recomputed after each outcome.

6. Async Processing Without an External Broker

The prototype uses a database-backed "poll due work" pattern (`DeliveryAttemptRepository.lockNextBatchDue` with `@Scheduled` workers) instead of Kafka, SQS, or RabbitMQ. This was a deliberate choice for the assignment's constraints.

Consideration	Assessment
Pros for this context	Zero extra infrastructure to run or explain; transactional consistency between "write the attempt" and "make it visible to the worker" comes for free; fully testable with in-memory H2; easy for a reviewer to reason about.
Cons at production scale	Polling latency floor equals the poll interval; a single logical queue table becomes a write hot-spot under high throughput; horizontal scale-out requires <code>SELECT ... FOR UPDATE</code> locking (implemented), which still contends on the same rows and index under very high concurrency.
Production path	Replace <code>DeliveryWorker</code> 's poll loop with a Kafka/SQS consumer that calls <code>DeliveryDispatcher.dispatch(attemptId)</code> directly, and have <code>DeliveryOrchestrationService</code> publish to the queue instead of relying on the next poll tick. Because <code>dispatch</code> already lives in a separate transactional bean, this swap does not touch <code>RoutingService</code> , <code>RetryPolicy</code> , or any channel provider.

7. Channel Routing Policy

Requested channels are filtered by recipient preference using an opt-out model. **CRITICAL** severity overrides opt-outs and fans out to every channel the platform supports, on the grounds that a safety-critical alert must not be silently suppressed by a stale preference.

See the `RoutingService` Javadoc and `docs/scenarios/01-greenfield.md` for the full policy and worked examples.

8. Retry and Failure Classification

Category	Failure types	Behaviour
Retryable	<code>TRANSIENT_PROVIDER_FAILURE</code> , <code>TIMEOUT</code> , <code>RATE_LIMITED</code>	Exponential backoff; honours a provider <code>Retry-After</code> hint when present.
Non-retryable	<code>PERMANENT_PROVIDER_REJECTION</code> , <code>INVALID_RECIPIENT</code> , <code>AUTH_ERROR</code>	Fail fast to <code>ABANDONED</code> .

Backoff is exponential with a 30-second base and 15-minute cap in production. Both values are externalised via `nms.retry.*` so integration tests can exercise the real retry path on a fast clock. See `RetryPolicy` and `FailureCategory` Javadoc for details.

9. Deduplication and Idempotency Boundaries

Two independent boundaries are enforced, both required by requirement 4.4.

9.1 Request-level

`idempotency_records` is unique on `(source_system, idempotency_key)`. Repeating a submission with the same key returns the original notification instead of creating a second logical one. Records are retained for 7 days, actively enforced by `IdempotencyCleanupJob` rather than merely documented (see [docs/scenarios/02-brownfield.md](#)).

9.2 Delivery-level

`delivery_attempts` is unique on `(notification_id, recipient_id, channel)`, combined with `DeliveryDispatcher`'s claim-then-dispatch split. Reprocessing a queued delivery cannot fan out into a duplicate send: claiming flips the row to `IN_PROGRESS` inside the same locked transaction that read it, and `dispatch()` re-checks status before sending.

10. Security and Data-Handling Notes

- Audit records (`AuditEvent.detail`) contain structured metadata only — channel, attempt counts, failure categories. Never raw message bodies, recipient PII beyond an opaque `recipientId`, or provider credentials (requirement 4.9).
- Bean Validation rejects malformed input at the API boundary before it reaches persistence.
- No secrets are hardcoded; the Postgres profile reads credentials from `NMS_DB_USERNAME` and `NMS_DB_PASSWORD` environment variables.
- `GlobalExceptionHandler` maps internal exceptions to typed API errors rather than leaking stack traces.

11. Deliberately Out of Scope

The following are called out explicitly rather than silently absent. See [docs/testing-strategy.md](#) for the full limitations and trade-offs list.

- **AuthN/AuthZ on the API** — would be fronted by the organisation's standard gateway / OAuth2 resource server in production; noted but not implemented to keep the prototype runnable standalone.
- **A durable message broker** — see Section 6.
- **Multi-tenant partitioning and per-source-system rate limiting.**
- **Real provider integrations** (SES, Twilio, FCM, Slack API) — providers are simulated deterministically.